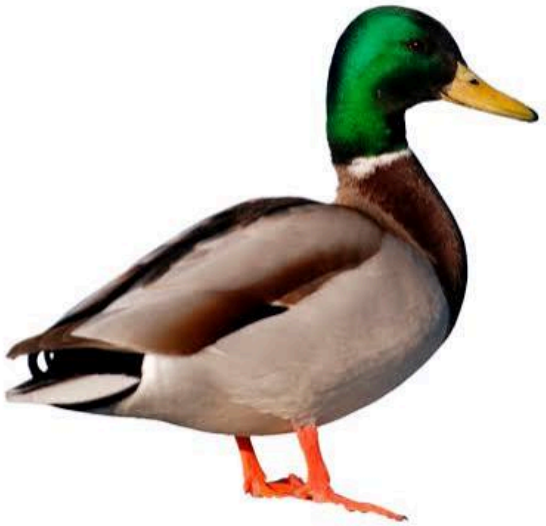
Ciência de Dados Aplicada ao Direito I

Aula 21

Quem ganha as ações de canabidiol



Prof. Bruno da Cunha de Oliveira - 2026/2

Reposição da aula de 28/10 e um atendimento

Vou marcar duas coisas na semana de 09/11: a reposição da aula de quarta, 28/10, com presença coletiva, e um horário de atendimento. Os horários seguem quatro regras do Insper:

#	Regra
1	sem conflito com aula, atendimento, monitoria ou atividade extra da turma
2	sem conflito com outra reposição, mesmo que ainda em análise
3	pelo menos 45 minutos de almoço entre 11h e 15h
4	pelo menos 15 minutos entre uma atividade e outra, para a locomoção

Horário da reposição

Opção	Dia	Horário	Duração
C	seg 09/11	16h30 às 18h30	2 horas
D	qui 12/11	9h45 às 11h45	2 horas
E	sex 13/11	7h30 às 9h30	2 horas

Horário do atendimento

Opção	Dia	Horário	Duração
A	seg 09/11	12h às 13h30	1h30
B	seg 09/11	12h30 às 14h	1h30
C	seg 09/11	16h30 às 18h30	2 horas
D	qui 12/11	9h45 às 11h45	2 horas
E	sex 13/11	7h30 às 9h30	2 horas

Problemas de funções e de classes

Quando	O quê
Agora	o notebook "Problemas de funções" está no Blackboard. Façam os exercícios 6, 10, 12 e 14
Qui, 05/11, 12h30	corrigimos esses quatro juntos no atendimento, na sala Peter Drucker, 2º andar do Prédio Claudio Haddad
Semana seguinte	"Problemas de classes": os exercícios 2, 8, 9 e 15, corrigidos no atendimento de 12/11
Qua, 25/11	simulado impresso, mais curto que a prova, corrigido em sala

No notebook, antes dos exercícios 1, 3 e 7: um exemplo resolvido

Devolver a própria condição

Uma comparação como `valor_sm <= 20` já é um valor: vale `True` ou `False`. Quando a saída da função é `True` ou `False`, a função pode devolver a própria condição, sem `if`. Na aula 16, o art. 171 do Código Penal virou função assim:

> Art. 171 - Obter, para si ou para outrem, vantagem ilícita, em prejuízo alheio, induzindo ou mantendo alguém em erro, mediante artifício, ardil, ou qualquer outro meio fraudulento:

Na aula, o erro era uma entrada só. Aqui ele se divide em duas, `induziu` e `manteve`, como na condição que vocês escreveram em grupo na aula 8, para a condição usar também o `or`.

In [ ]:
```
def configura_estelionato(vantagem, prejuizo, induziu, manteve, fraude):
    return vantagem and prejuizo and (induziu or manteve) and fraude


print(configura_estelionato(True, True, False, True, True))
print(configura_estelionato(True, True, False, False, True))
print(configura_estelionato(True, False, True, False, True))
```

Saída:

```
True
False
False
```

Se travarem: uma dica de cada vez

Dicas. Abra uma de cada vez, só quando travar, e tente de novo antes de abrir a seguinte.

▼ Dica 1

Separe o caput e o § 3º em exigências. Quantas são, e cada uma é sobre qual fato da pessoa?

▼ Dica 2

A exigência sobre quem pode pedir admite duas formas. Basta uma delas, ou precisa das duas? E a exigência da renda vale junto com ela ou no lugar dela? Cada resposta escolhe um conectivo.

▶ Dica 3

#	Antes de abrir a dica
1	separem o artigo em exigências; cada uma vira um parâmetro
2	testem um caso que deve dar `True` e outro que deve dar `False`

Onde paramos: um cliente, três perguntas

Insper

Uma empresa quer entrar no Brasil

Uma empresa vende produtos médicos à base de canabidiol e quer começar a vender no Brasil. Boa parte do acesso a medicamentos aqui passa pela Justiça, e ela quer entender isso antes de decidir.

#	O que ela quer saber
1	As pessoas judicializam mais contra entes públicos ou contra privados?
2	Dentro de cada grupo, quem é mais acionado?
3	Quem ganha e quem perde?

300 sentenças sobre canabidiol

O quê	Como
Busca	"canabidiol" no banco de sentenças do TJSP, com o juscraper, sem outro filtro
Resultados	4.618 sentenças, em 462 páginas
No CSV	300: 30 páginas sorteadas, 10 sentenças por página
Colunas	`classe`, `assunto`, `comarca`, `data` e `texto`

Nenhuma coluna diz quem é o réu ou quem ganhou. As duas informações estão dentro de `texto`.

Abrir a base e ler o começo

No caderno da aula 21, os passos 1 e 2: abrir `decisoes-canabidiol-tjsp.csv` no pandas, contar as linhas e imprimir os 200 primeiros caracteres dos cinco primeiros textos.

300 linhas, e o começo de cada texto

RESPOSTA

```
import pandas as pd

decisoes = pd.read_csv("decisoes-canabidiol-tjsp.csv")
print(len(decisoes))
for texto in decisoes["texto"][:5]:
    print(texto[:200])
    print("----------")
```

A primeira abre com o cabeçalho

```
SENTENÇA
Processo Digital nº:  1500375-25.2024.8.26.0344
Classe - Assunto  Petição Cível - Petição intermediária
Requerente: Braz Bento dos Santos
Requerido:  Fazenda Pública do Estado de São Paulo e ou
```

A terceira não

```
TERMO DE CONCLUSÃO
Eu,           JHENYFFER SANTINA MARIA REIS MORES, Estagiário
Nível Superior, matr. nº E50318838, em 26 de fevereiro de
2024, faço estes autos conclusos ao(à) MM. Juiz(a) de Direito
```

Das cinco, duas abrem com `Requerente:` e `Requerido:`; três não. Cortar uma posição fixa do começo não serve: o cabeçalho nem sempre está lá, e o nome do réu muda de tamanho.

O padrão, e o que coletamos

`Requerido:`, e depois o nome, até o fim da linha.

Coletamos só o requerido. O requerente é uma pessoa doente, e o nome junto da doença é dado pessoal de saúde; para o cliente, importa contra quem se pede.

Três aspas: um texto longo dentro do código

```
trecho = """SENTENÇA

Processo nº: 0004177-52.2023.8.26.0664
Requerido: Fazenda Pública do Estado de São Paulo"""
print(trecho)
print("Requerido:" in trecho)
```

Com três aspas, o texto pode ter várias linhas, e as quebras ficam guardadas. Serve para colar o começo de uma sentença e testar um padrão nele. A regex do resultado, mais adiante, também vem entre três aspas.

Basta procurar o padrão

Peçam ao assistente

Uma expressão regular (regex) descreve um padrão de texto. No caderno, passo 5, peçam ao assistente do VS Code:

> "Uma expressão regular que pegue, em cada texto da coluna `texto`, o que vem depois de `Requerido:` até o fim da linha, e guarde numa coluna nova `requerido`."

Rodem e contem quantas linhas ganharam valor.

A que eu recebi, com acerto de 269 de 300

RESPOSTA

```
decisoes["requerido"] = decisoes["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
print(decisoes["requerido"].notna().sum())
print(decisoes["requerido"][:5])
```

Ler o padrão que o assistente escreveu

Pedaço	O que diz
`Requerid[oa]s?:`	Requerido, Requerida, Requeridos ou Requeridas, e os dois-pontos
`\s*`	espaços ou tabulação, quantos houver
`([^\n]+)`	tudo até a quebra de linha; os parênteses marcam o que guardar

Não é para escrever: é para conferir se o que o assistente entendeu é o que vocês pediram.

31 ficaram vazias

```
269
0       Fazenda Pública do Estado de São Paulo e outro
1    UNIMED DE GUARULHOS COOPERATIVA DE TRABALHO ME...
2    Fazenda Pública do Estado de São Paulo e INSTI...
3                                                  NaN
4                              UNIMED NACIONAL e outro
Name: requerido, dtype: str
```

`str.extract` guarda o trecho que corresponde ao padrão; onde não há, fica `NaN`, o vazio. As 31 não usam `Requerido:` (`Impetrado`, `Executado`, sem cabeçalho).

Dá para reduzir as 31?

Peçam ao assistente uma segunda rodada: "31 ficaram vazias; olhe os textos sem valor e amplie o padrão para os outros rótulos do polo passivo". Rodem e contem de novo.

Quantas sobram? E por que a regra sempre vai deixar alguma?

288 de 300, e as 12 que sobram

RESPOSTA

```
padrao = (r"(?:Requerid|Impetrad|Executad|Reclamad)"
          r"[oa]s?:\s*([^\n]+)")
decisoes["requerido"] = decisoes["texto"].str.extract(
    padrao)
print(decisoes["requerido"].notna().sum())
```

Impetrado, Executado e Reclamado entram; sobram 12 sem cabeçalho nenhum.
O padrão cresce a cada rodada, e sempre há um texto que ele não descreve.

O padrão, peça por peça

Animação no navegador: `animacao-regex-reu.html`, cenário `tres-sentencas`.

Quem é mais acionado?

Com a coluna `requerido` pronta, `value_counts` conta quantas vezes cada valor aparece:

```
print(decisoes["requerido"].value_counts().head(6))
```

Olhem a lista que sai. Ela responde à pergunta 2 do cliente?

A mesma Fazenda, escrita de três jeitos

RESPOSTA

```
requerido
Fazenda Pública do Estado de São Paulo                  68
Fazenda Pública do Estado de São Paulo e outro          28
Sul América Companhia de Seguro Saúde                    9
PREFEITURA MUNICIPAL DE JABOTICABAL e outro              8
UNIMED DE GUARULHOS COOPERATIVA DE TRABALHO MEDICO       4
PREFEITURA MUNICIPAL DE SÃO PAULO                        4
Name: count, dtype: int64
```

Como vocês lidariam com isso?

A mesma Fazenda aparece escrita de três jeitos, e prefeituras vêm ora em caixa alta, ora não. Contada assim, a lista divide o mesmo réu em várias linhas.

Como vocês lidariam com isso? Quais estratégias usariam?

Contar ainda não é responder

RESPOSTA

O que a lista mostra	Uma estratégia
a mesma Fazenda com e sem "e outro"	um dicionário: cada grafia aponta para um nome só
PREFEITURA em caixa alta	comparar tudo com `.upper()`
"e outro", "e outros"	cortar o texto antes de "e outro"

Padronizar é a aula de limpar uma base. Hoje a resposta ao cliente leva essa ressalva.

Público ou privado

Insper

Como dividiriam?

Temos a coluna `requerido`, com Fazenda Pública, prefeituras, Sul América, Unimed, Amil, Bradesco Saúde. Como vocês dividiriam esses valores entre público e privado, numa regra que o programa aplique às 300 linhas?

Uma regra por palavras

RESPOSTA

```
publico = []
for requerido in decisoes["requerido"]:
    nome = str(requerido).upper()
    if ("FAZENDA" in nome or "ESTADO" in nome
            or "MUNIC" in nome or "PREFEITURA" in nome):
        publico.append(True)
    else:
        publico.append(False)
```

A regra, linha por linha

Animação no navegador: `animacao-publico-privado.html`, cenário `regra`.

O que a regra diz

```
decisoes["publico"] = publico
print(decisoes["publico"].value_counts())
```

Um `for` sobre a coluna, um `if` com `in` por palavra, e a lista `publico` vira coluna nova. `str()` porque o vazio não é texto; `.upper()` para "Prefeitura" e "PREFEITURA" contarem igual; `MUNIC` pega Município e Municipal.

196 públicos, e o resto não é tudo privado

```
publico
True     196
False    104
Name: count, dtype: int64
```

Entre as 300: 196 com ente público; 92 sem nenhuma das palavras, que a regra chama de privado; e as 12 sem réu, que viraram "nan" e caíram no `False` junto com elas.

Primeira resposta ao cliente, com ressalva

O cliente nos perguntou: as pessoas judicializam mais contra entes públicos ou contra privados? Com a regra, já dá para responder:

> Nesta base, quase dois terços das ações (196 de 300) são contra o Estado ou um município.

A ressalva é o termo de busca: "canabidiol" puxa quem pede ao SUS. Contra um plano de saúde, talvez se escreva de outro jeito, e essas sentenças ficaram fora da coleta.

Quem ganha

Quem ganha e quem perde?

A terceira pergunta do cliente: quem ganha e quem perde? Já temos o réu numa coluna. Falta o resultado de cada sentença.

Como vocês fariam isso? Onde, no texto, está o resultado, e como o programa o encontraria?

Procurar "procedente"

```
procedentes = 0
improcedentes = 0
for texto in decisoes["texto"]:
    if "procedente" in texto:
        procedentes = procedentes + 1
    if "improcedente" in texto:
        improcedentes = improcedentes + 1
print(procedentes)
print(improcedentes)
```

Sai 165 e 68. O que está errado nessa conta?

165 inclui os 68

RESPOSTA

```
165
68
```

"improcedente" contém "procedente": toda sentença que diz "improcedente" entra também na conta de "procedente". Os 165 incluem os 68. E "PROCEDENTE" em caixa alta escapa das duas contas.

Procurar uma palavra solta não responde "quem ganhou". O padrão precisa dizer o que vem antes e depois dela.

A melhor regex que o assistente conseguiu

Pedi: "a melhor expressão regular possível para dizer se a sentença julgou procedente, parcialmente procedente, improcedente ou extinguiu sem mérito". Voltou isto, em quatro grupos, entre três aspas. Primeiro olhamos os grupos; depois, lemos peça por peça:

```
PADRAO = re.compile(r"""
    (?P<parcial>
        julgo\s+(?:\w+\s+){0,3}?
        (?:parcialmente\s+procedentes?
          |procedentes?\s+em\s+parte)
      | parcial\s+procedência)
```

... o grupo improcedente ...

```
  | (?P<improcedente>julgo\s+(?:\w+\s+){0,3}?improcedentes?
      | denego\s+a\s+(?:segurança|ordem)
      | rejeito\s+o\s+pedido
      | nego\s+provimento)
```

... o grupo procedente ...

```
  | (?P<procedente>julgo\s+(?:\w+\s+){0,3}?procedentes?
      (?!\s+em\s+parte)
    | concedo\s+a\s+(?:segurança|ordem)
    | acolho\s+o\s+pedido)
```

... e o grupo extinto

```
   | (?P<extinto>julgo\s+extint[oa]s?\b[^.]{0,160}?
        sem\s+(?:resolução|julgamento|exame|análise)
        \s+d[oe]\s+mérito
      | extingo\s+o\s+(?:processo|feito)[^.]{0,160}?
        sem\s+(?:resolução|julgamento)\s+d[oe]\s+mérito
      | art(?:igo|\.)?\s*485
      | homologo\s+(?:a\s+desistência|o\s+acordo
                    |a\s+transação))
""", re.IGNORECASE | re.VERBOSE)
```

Lendo a regex: palavras e repetições

Peça	O que diz
`julgo`	a palavra; com `re.IGNORECASE`, em caixa alta ou não
`\s+`	um ou mais espaços ou quebras de linha
`(?:\w+\s+){0,3}?`	de zero a três palavras no meio: "julgo o pedido procedente"
`procedentes?`	o ? torna o "s" opcional
`[^.]{0,160}?`	até 160 caracteres, sem passar de um ponto

Lendo a regex: grupos e escolhas

Peça	O que diz
`a | b`	uma coisa ou outra: "denego a segurança" ou "rejeito o pedido"
`(?: ... )`	junta peças num bloco, sem guardar nada
`(?P<parcial> ... )`	dá nome ao grupo; o nome do grupo que casou vira o valor da coluna
`(?!\s+em\s+parte)`	só casa se NÃO vier "em parte" logo depois
`re.VERBOSE`	deixa escrever o padrão em várias linhas, com espaços, para ficar legível

Uma frase passando pelo padrão

> "Ante o exposto, julgo PARCIALMENTE PROCEDENTE o pedido."

Grupo	O que acontece
`parcial`	casa com `julgo` e `parcialmente\s+procedentes?`: a coluna recebe "parcial"
`procedente`	nem chega a ser testado: o primeiro grupo que casa vence

Por isso `parcial` vem antes de `procedente`: na ordem inversa, "parcialmente procedente" contaria como vitória inteira.

A função que aplica o padrão

```
def resultado(texto):
    achados = list(PADRAO.finditer(texto[-3000:]))
    if not achados:
        return "?"
    return achados[-1].lastgroup

decisoes["resultado"] = decisoes["texto"].apply(resultado)
print(decisoes["resultado"].value_counts())
```

Procura só no fim do texto, onde fica o dispositivo, e fica com a última frase que casou. Sem frase nenhuma, "?".

40 que ela não alcança

RESPOSTA

```
resultado
procedente      130
improcedente     88
?                40
parcial          27
extinto          15
Name: count, dtype: int64
```

Quatro grupos de frases, e 40 de 300 sem classe.

Complexa, e ainda assim incompleta

O que a regex faz	O que escapa
procura nos últimos 3.000 caracteres	dispositivo mais acima
quatro grupos de frases: julgo, concedo, denego, extingo	juiz que escreve de outro jeito
fica com a última frase que casou	sentença que julga dois pedidos

A regra acerta onde o texto segue o padrão que eu imaginei. Para saber quantas estão certas, alguém lê uma amostra.

Outra maneira: utilizando Inteligência Artificial

Quem mais poderia preencher as três colunas?

Em vez de descrever o padrão do texto, dá para pedir a alguém que leia cada sentença e responda: quem é o réu, se é público ou privado, qual foi o resultado.

Ler 300 à mão leva um dia. Quem mais lê texto e responde em formato fixo?

Um modelo de linguagem

RESPOSTA

Um modelo de linguagem lê a sentença e preenche o formulário. Chamado de dentro do programa, ele faz isso para cada linha da tabela. Hoje vamos só usar; como ele funciona por dentro e o que custa ficam para as próximas aulas.

Para que o modelo de linguagem faça isso, precisamos de três coisas: uma chave de acesso a um serviço que roda o modelo, um formulário com as perguntas que queremos fazer, e a biblioteca que liga os dois à tabela.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

Hoje a chave vai colada no programa, e isso não é seguro: como guardá-la fora do código é assunto de uma aula adiante. Não mandem a chave a ninguém, nem para mim.

Ler o README de uma biblioteca

A biblioteca é o dataframeit, no GitHub. O README de uma biblioteca costuma ter o que basta para a primeira chamada: como instalar, um exemplo mínimo e a lista de argumentos.

```
uv add "dataframeit[groq]"
```

Vamos ler juntos o exemplo do README e adaptá-lo às nossas três perguntas.

O formulário

```
from typing import Literal
from pydantic import BaseModel
class Formulario(BaseModel):
    requerido: str
    polo_passivo: Literal["público", "privado"]
    resultado: Literal["procedente",
                       "parcialmente procedente",
                       "improcedente",
                       "extinto sem julgamento do mérito"]
```

O que cada linha da classe diz

Um campo por linha: nome, dois-pontos e tipo. `Literal` fecha a lista de valores que o modelo pode devolver; `str` é texto livre. A classe herda de `BaseModel` o trabalho de conferir cada resposta.

Uma chamada para dez sentenças

```
from dataframeit import dataframeit
dez = decisoes.head(10)
tabela = dataframeit(dez, Formulario,
    "Leia a sentença e preencha os campos: {texto}",
    provider="groq", text_column="texto",
    api_key="gsk_...")
print(tabela[["requerido", "polo_passivo", "resultado"]])
```

`head(10)` separa as dez primeiras das 300, para a chamada ser curta. Uma linha de cada vez: o texto vai a um servidor da Groq, o modelo responde, a classe confere. O caderno grava `tabela` em `dez-decisoes-modelo.csv`.

Os argumentos do dataframeit

Argumento	O que é
`dez`	as dez primeiras sentenças, uma por linha
`Formulario`	o formulário, que confere cada resposta
o texto com `{texto}`	o pedido ao modelo; cada sentença entra no lugar de `{texto}`
`provider="groq"`	o serviço que roda o modelo
`text_column="texto"`	a coluna em que está o texto
`api_key="gsk_..."`	a sua chave, colada hoje no programa

Rodem nas dez

Com a chave no lugar de `gsk_...`, rodem a célula. Leva alguns segundos por sentença.

Deu certo quando `tabela` tem dez linhas e três colunas novas, cada uma com um valor que a classe aceitou. Comparem com o que a regex deu para as mesmas dez.

Os dois caminhos lado a lado

Guardar antes de comparar: `to_csv`

```
dez = decisoes.head(10)
dez["requerido_regex"] = dez["texto"].str.extract(
    r"Requerid[oa]s?:\s*([^\n]+)")
dez.to_csv("dez-decisoes-regex.csv", index=False)
```

`to_csv` grava a tabela num arquivo; `index=False` deixa de fora a numeração das linhas. O caderno gravou `tabela`, a do modelo, do mesmo jeito. Com os dois arquivos salvos, a comparação não depende de rodar tudo de novo.

As mesmas dez, regex contra modelo

```
import pandas as pd

regex = pd.read_csv("dez-decisoes-regex.csv")
modelo = pd.read_csv("dez-decisoes-modelo.csv")

regex["requerido_modelo"] = modelo["requerido"]
print(regex[["requerido_regex", "requerido_modelo"]])
```

`read_csv` relê os dois arquivos. Onde as duas colunas diferem, quem acertou?

Quando usar cada um?

Nas dez sentenças, comparem os resultados e discutam em dupla:

#	Pergunta
1	Em que tarefa a regex basta, e por quê?
2	Em que tarefa o modelo é melhor, e o que ele custa?
3	Como vocês saberiam quantas respostas do modelo estão certas?

Regra para o que tem forma, modelo para o que tem sentido

RESPOSTA

	Expressão regular	Modelo de linguagem
acerta	em texto de forma fixa	em texto variado
custa	testar a regra	uma chamada por linha
erra	em silêncio	com confiança
se confere	lendo o vazio	lendo uma amostra

O que ainda vamos aprender

Etapa	A pergunta que ela responde
definir a amostra	que termo, classe, período e tribunal, e por quê
o que coletar e o que não	que dado de pessoa fica de fora, e com que base
coletar	raspar e guardar, com a ficha do que foi feito
limpar	a mesma Fazenda com um nome só; vazios e duplicados
descrever	contar, agrupar e cruzar: público contra privado, quem ganha em cada grupo
visualizar	o gráfico que o cliente lê sem a tabela

Na próxima aula

A regra contra o modelo de linguagem nas mesmas sentenças, até saber quando usar cada um. Depois, um cliente novo, uma grande companhia aérea, e as perguntas de pesquisa que valem a pena fazer por ela.